

DIGITAL FORENSICS

FINAL INVESTIGATION REPORT

MD5 Hash Recovery

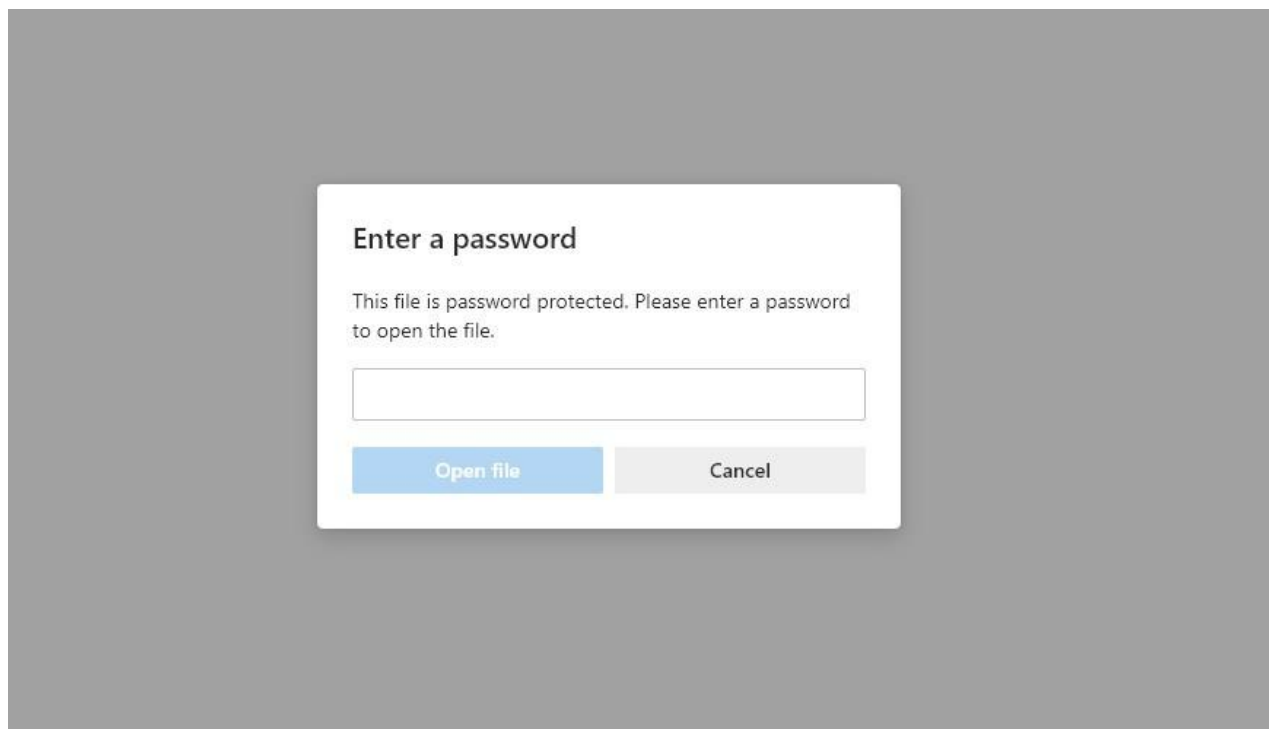
Case Reference No.	DFIR-2026-005
Investigator	Julian Derry
Date of Analysis	February 15, 2026
Prepared For	Public
Classification	Public

Table of Contents

Table of Contents	
1. Overview.....	
2. Objective.....	
3. Initial Tool Selection	
4. PDF Hash Extraction	
5. Hash Preservation.....	
6. Hash Mode Identification	
7. PDF Version Identification	
8. Hashcat Testing.....	
9. Password Recovery.....	
10. Verification	
11. Tools Used	
12. Lessons Learned.....	
13. Conclusion	

1. Overview

This exercise involved recovering and cracking the password hash from a protected PDF file named logs_protected.pdf. The file was provided by HIVE CONSULT as a practical password-recovery challenge. The investigation documented the full workflow from initial tool selection through successful password recovery. During the process, several important lessons were learned regarding file-type identification, correct hash extraction methods, shell handling of special characters, and PDF hash-mode selection in Hashcat.



2. Objective

The objective was to extract the password hash from the protected PDF, determine the appropriate cracking mode, recover the password using a wordlist attack, and verify successful access to the document.

3. Hash Identification

I first attempted to extract a hash using:

office2john logs_protected.pdf

The command returned: **logs_protected.pdf : Invalid OLE file**

This demonstrated that office2john is intended for Microsoft Office documents such as Word, Excel, and PowerPoint files, not PDF files.

```
(kali@kali)-[~/Desktop]
$ office2john logs_protected.pdf
logs_protected.pdf : Invalid OLE file
```

4. PDF Hash Extraction

After researching PDF-specific extraction methods, I identified pdf2john.py as the correct tool.

Command: pdf2john.py logs_protected.pdf

The command successfully extracted the PDF password hash.

```
(kali@kali)-[~/Desktop]
$ pdf2john logs_protected.pdf
logs_protected.pdf:$pdf$2*3*128*-4*1*16*89299b42e062b748b639e4bac7a7cd80*32*35b9b48882f18a5c05098bb6309bd99f28bf4e5e4e758a4164004e56ffa0108*32*7ea00003b4a2f2305197159632f11ae613b8ea6e4466cdc5dd41abd31730d9bb
```

5. Hash Preservation

The extracted hash was saved to a text file for reuse during cracking.

Command: echo 'HASH_VALUE' > pdflogs.txt

```
(kali@kali)-[~/Desktop]
$ echo '$pdf$2*3*128*-4*1*16*89299b42e062b748b639e4bac7a7cd80*32*35b9b48882f18a5c05098bb6309bd99f28bf4e5e4e758a4164004e56ffa0108*32*7ea00003b4a2f2305197159632f11ae613b8ea6e4466cdc5dd41abd31730d9bb' > pdflogs.txt
```

I intentionally used single quotes because characters such as \$ and * may be interpreted by the shell when double quotes are used.

To verify the saved hash, I used: **cat pdflogs.txt**

```
(kali@kali)-[~/Desktop]
$ cat pdflogs.txt
$pdf$2*3*128*-4*1*16*89299b42e062b748b639e4bac7a7cd80*32*35b9b48882f18a5c05098bb6309bd99f28bf4e5e4e758a4164004e56ffa0108*32*7ea00003b4a2f2305197159632f11ae613b8ea6e4466cdc5dd41abd31730d9bb
```

I also learned that the entire hash string generated by pdf2john must be saved, not only the hexadecimal portion.

6. Hash Mode Identification

To identify supported PDF hash modes in Hashcat, I used: **hashcat --help | grep -i pdf**

This displayed the available PDF hash modes.

```
(kali㉿kali)-[~/Desktop]
$ hashcat --help | grep -i pdf
10400 | PDF 1.1 - 1.3 (Acrobat 2 - 4) | Document
10410 | PDF 1.1 - 1.3 (Acrobat 2 - 4), collider #1 | Document
10420 | PDF 1.1 - 1.3 (Acrobat 2 - 4), collider #2 | Document
10500 | PDF 1.4 - 1.6 (Acrobat 5 - 8) | Document
25400 | PDF 1.4 - 1.6 (Acrobat 5 - 8) - user and owner pass | Document
10600 | PDF 1.7 Level 3 (Acrobat 9) | Document
10700 | PDF 1.7 Level 8 (Acrobat 10 - 11) | Document
```

7. PDF Version Identification

I initially believed the correct mode was 10400, but further testing showed that assumption was incorrect.

To identify the PDF encryption details, I used: **pdftinfo logs_protected.pdf**

I researched the relationship between the algorithm version and revision values and the corresponding Hashcat mode.

The version information is in the **first two numeric fields** after `$pdf$`.

So here:

powershell

 Copy code

```
$pdf$2*3*128*...
```

```
↑ ↑
```

```
| |
```

```
| └─ R (revision)
```

```
└─ V (algorithm version)
```

So in your case:

- `V = 2`
- `R = 3`

What that means

In PDF encryption:

- `V` = algorithm version
- `R` = security handler revision
- `128` = key length in bits

With `v=2` and `R=3`, this corresponds to:

- 128-bit RC4 encryption
- Used in Acrobat 5–6 era PDFs
- Compatible with PDF 1.4 / 1.5 encryption models

This is older encryption. Not modern AES-256.

If this were something stronger like AES-256, you'd see higher V/R values (like V=5, R=6).

8. Hashcat Testing

Because the initially selected mode did not work, I tested the hash against the different PDF modes returned by:

hashcat --help | grep -i pdf

This confirmed that choosing the wrong mode prevents recovery even when the correct password exists in the wordlist.

9. Password Recovery

The hash was eventually cracked using a different PDF hash mode.

Example command: hashcat -m [HASH_MODE] pdf_hash.txt wordlist.txt

To display the recovered password, I used: hashcat -m [HASH_MODE] pdf_hash.txt --show

```
(kali@kali)-[~/Desktop]
└─$ hashcat -m 10420 pdflogs.txt /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, LLVM 17.0.6, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
=====
* Device #1: cpu-haswell-13th Gen Intel(R) Core(TM) i5-13420H, 4203/8470 MB (2048 MB allocatable), 4MCU

/usr/share/hashcat/OpenCL/m10420_a0-optimized.cl: Pure kernel not found, falling back to optimized kernel
Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 32

Hashfile 'pdflogs.txt' on line 1 ($pdf$2...13b8ea6e4466cdc5dd41abd31730d9bb): Separator unmatched
No hashes loaded.

Started: Tue Feb 17 05:22:14 2026
Stopped: Tue Feb 17 05:22:14 2026

(kali@kali)-[~/Desktop]
└─$ hashcat -m 104500 pdflogs.txt /usr/share/wordlists/rockyou.txt
Invalid --hash kernel value specified

$pdf$2*3*128*-4*1*16*89299b42e062b748b639e4bac7a7cd80*32*35b9b48882f18a5c05098bb6309bd99f28bf4e5e4e758a4164004e56fffa0108*32*7ea00003b4a2f2305
197159632f11ae613b8ea6e4466cdc5dd41abd31730d9bb:computer

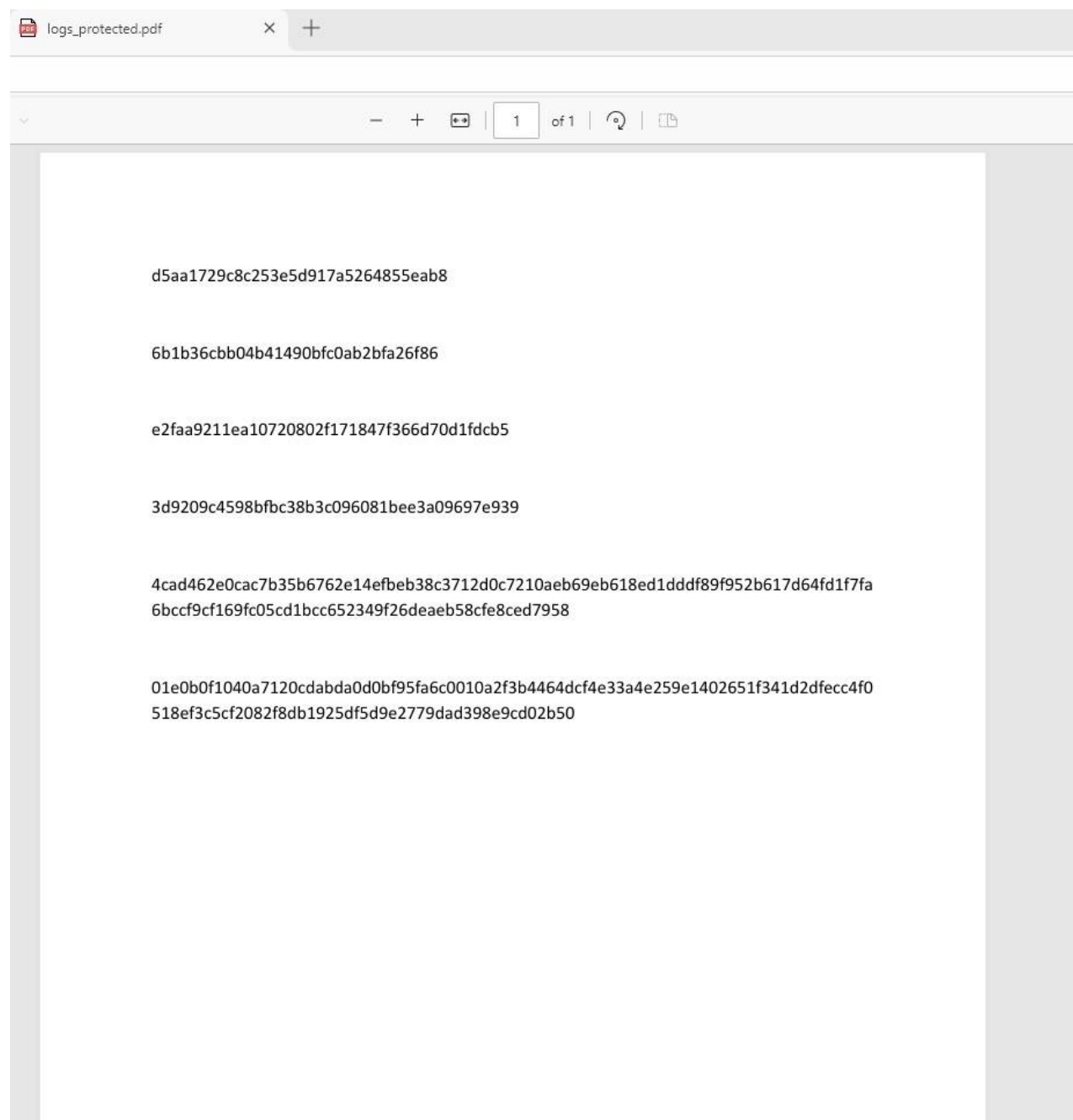
Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 10500 (PDF 1.4 - 1.6 (Acrobat 5 - 8))
Hash.Target.....: $pdf$2*3*128*-4*1*16*89299b42e062b748b639e4bac7a7cd...30d9bb
Time.Started.....: Tue Feb 17 05:23:35 2026 (1 sec)
Time.Estimated...: Tue Feb 17 05:23:36 2026 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 13054 H/s (12.07ms) @ Accel:256 Loops:70 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 1024/14344385 (0.01%)
Rejected.....: 0/1024 (0.00%)
Restore.Point...: 0/14344385 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-70
Candidate.Engine.: Device Generator
Candidates.#1....: 123456 -> bethany
Hardware.Mon.#1..: Util: 22%

Started: Tue Feb 17 05:22:39 2026
Stopped: Tue Feb 17 05:23:38 2026

(kali@kali)-[~/Desktop]
```

10. Verification

The recovered password successfully opened logs_protected.pdf.



11. Tools Used

pdf2john.py, Hashcat, pdftinfo

12. Lessons Learned

- File type must be identified before selecting a hash-extraction tool.

- office2john is not suitable for PDF files.
- pdf2john.py is the correct extraction tool for protected PDFs.
- Save the full extracted hash, not a partial value.
- Use single quotes when saving hashes containing shell-sensitive characters.
- Verify saved hashes before cracking.
- Selecting the correct Hashcat mode is critical.
- pdftinfo can assist with identifying PDF encryption details.
- Practical testing may still be required when mode identification is uncertain.

13. Conclusion

The password-recovery exercise was completed successfully. The protected PDF hash was extracted, preserved correctly, tested against multiple PDF hash modes, cracked with Hashcat, and the recovered password was verified by opening the document. The investigation reinforced the importance of correct tool selection, careful hash handling, and accurate mode identification during password-recovery operations.